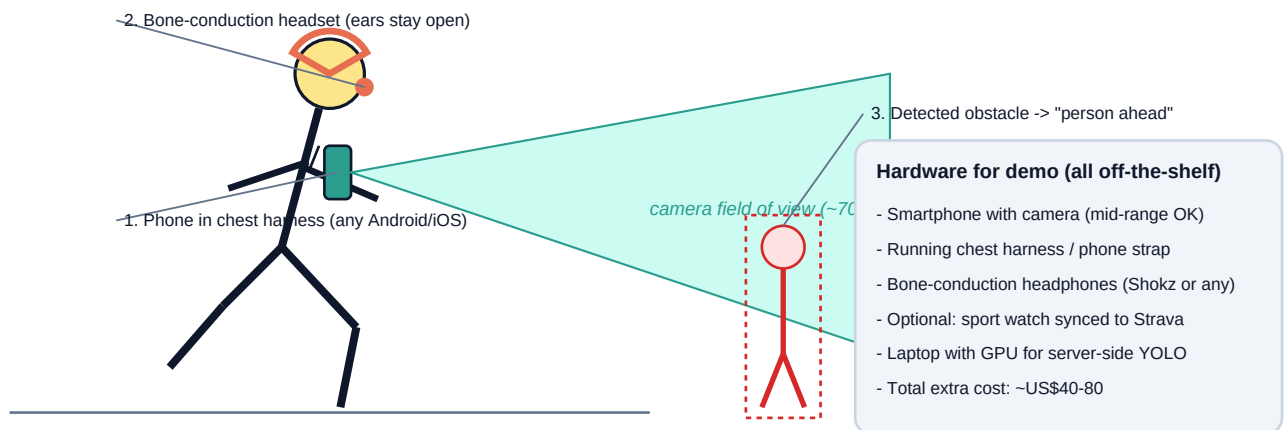


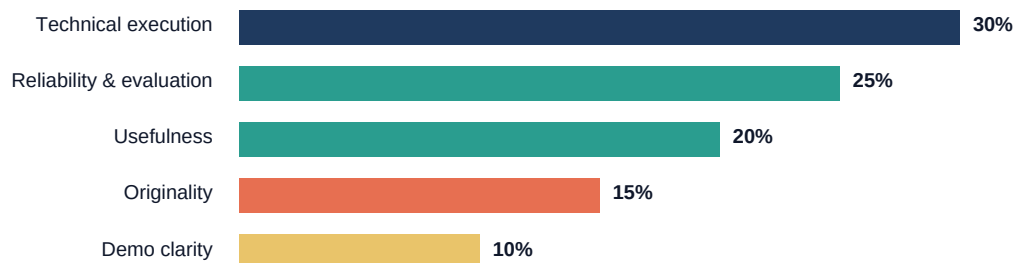
RunSense

An agentic training coach and obstacle-alert layer for blind and low-vision runners

Hackathon technical plan: problem, novelty, architecture, technology stack, evaluation strategy, build timeline and resources



Challenge brief: *Build one useful, multi-step AI agent. Connect it to at least three external apps. Show how you know it works.*



Judging weights this plan is designed around

Prepared September 2026 - v1.0

Contents

1. Executive summary
 2. Why this matters (importance)
 3. Landscape and novelty
 4. Solution overview and rubric mapping
 5. System architecture
 6. Perception module (computer vision)
 7. Agent core (multi-step orchestration)
 8. External integrations (the 3+ apps)
 9. Frontend
 10. Backend and data model
 11. Reliability and evaluation plan
 12. Safety, ethics and known limitations
 13. 48-hour build plan
 14. Demo script (5 minutes)
 15. Resources and references
- Appendix A - Code skeletons
- Appendix B - Risk register

1. Executive summary

RunSense is a single AI agent that owns a blind runner's training life end to end: it pulls real run data from Strava, reasons about training load, writes adaptive sessions into Google Calendar (inviting a guide runner when one is needed), delivers every instruction as speech through bone-conduction headphones, and - during the run itself - adds a lightweight computer-vision alert layer that tells the runner **where** an obstacle is ("person left", "stop, person ahead") in under 400 ms.

The agent is multi-step by design: **plan** (read data, reason) -> **act** (schedule, notify, coach) -> **observe** (run session, log alerts, verify the upload) -> **adapt** (compare plan vs actual, rewrite next week). Every tool call is traced, and a clip-based evaluation harness produces hard numbers for detection recall, zone accuracy and latency. That is our answer to "show how you know it works".

Scope decision that makes this buildable in 48 hours: RunSense does not claim to replace a guide runner or solve untethered navigation (Google DeepMind's Running Guide agent is attacking that with custom hardware). It is the **coaching and awareness layer that is missing from every existing tool**, and it works with a guide, on a track, or on a known loop today.

2. Why this matters (importance)

- The WHO estimates roughly 2.2 billion people live with a near or distance vision impairment, and the IAPB Vision Atlas counts around 43 million people who are blind. Physical inactivity in this population is well documented and drives secondary health problems.
- Running is one of the most accessible sports for blind athletes (no ball, no court, low equipment cost), yet participation is bottlenecked by the availability of trained sighted guides. Every session needs a person who is free at that exact time and can match pace.
- Existing assistive tech focuses on **the run itself** (staying on a line, avoiding a pole). Nobody helps with the 90% of the sport that happens between runs: what to do today, how hard, is it safe to progress, who is guiding, did the session actually happen.
- Coaches are expensive and rarely trained in accessibility; mainstream fitness apps are visual-first (charts, maps, taps), which makes them close to unusable with a screen reader mid-run.

RunSense therefore attacks three concrete failure points: **plan quality** (adaptive, data-driven, safe progression), **logistics** (guide scheduling and confirmation happen automatically), and **in-run awareness** (spoken obstacle direction as a second pair of eyes alongside the guide).

3. Landscape and novelty

Existing tool	What it does	What it does not do
Google Project Guideline (2020-)	Phone camera follows a painted line; tones steer the runner back to centre. On-device, offline.	Needs painted lines and a facilitator. No training plan, no scheduling, no obstacle semantics.
Google DeepMind Running Guide agent (May 2026)	Chest-mounted Pixel with spatial reasoning; DANGER / WARNING / NOTICE alerts; tested with SG Enable.	Research prototype on custom silicon, not a public API. Zero coaching, logistics or adaptation.
Meta / Oakley AI glasses	General-purpose scene description on request.	Not real-time, not running-specific, no alert prioritisation, no planning.
Strava, Garmin Coach, Runna	Adaptive plans for sighted runners.	Visual UI; no accessibility-aware constraints (guide availability, safe venues); no in-run awareness.

Novelty claims (what a judge should write down)

- **First agent that closes the loop** across data (Strava) -> plan (LLM) -> logistics (Calendar + guide) -> delivery (TTS) -> in-run perception -> re-planning. Each existing tool covers one box.
- **Accessibility-aware planning constraints** are first-class: a hard interval session is only scheduled if a guide accepted the invite or the venue is a track/treadmill; otherwise the agent swaps in a safe alternative and says why.

- **Voice-first by construction:** every output is a sentence a TTS engine can speak; no state exists only in a chart.
- **Honest, measurable perception:** we reproduce the DANGER/WARNING/NOTICE idea with off-the-shelf YOLO on any phone, and we publish recall, zone accuracy and latency instead of claiming untethered safety.
- **Positioning:** complementary to Guideline / Running Guide - they keep you on the path, RunSense decides what today's run is and adds a second pair of eyes.

4. Solution overview and rubric mapping

Criterion (weight)	How RunSense scores it	Evidence shown in demo
Technical execution (30%)	Real OAuth integrations, streaming perception on device, LLM tool-use loop with retries and idempotent writes, tracked objects with velocity.	Live camera alerts; live calendar write; code walkthrough of orchestrator.
Reliability & evaluation (25%)	Golden clip set (40 clips) with labelled zones; plan-sanity rule tests over 10 synthetic athletes; per-tool-call traces; latency budget enforced in CI.	Eval dashboard with numbers; replay of one trace; a deliberately failed API call auto-recovered.
Usefulness (20%)	Solves guide logistics + progression + in-run awareness for an underserved group; works with existing hardware.	Persona walkthrough; letter of intent / feedback from a blind runner or club if obtainable.
Originality (15%)	Closed-loop agent + accessibility constraints; complementary to big-tech perception work.	Landscape slide with the gap highlighted.
Demo clarity (10%)	One persona, one week, one run - shown in 5 minutes with audio the judges hear.	Scripted demo (section 14).

5. System architecture

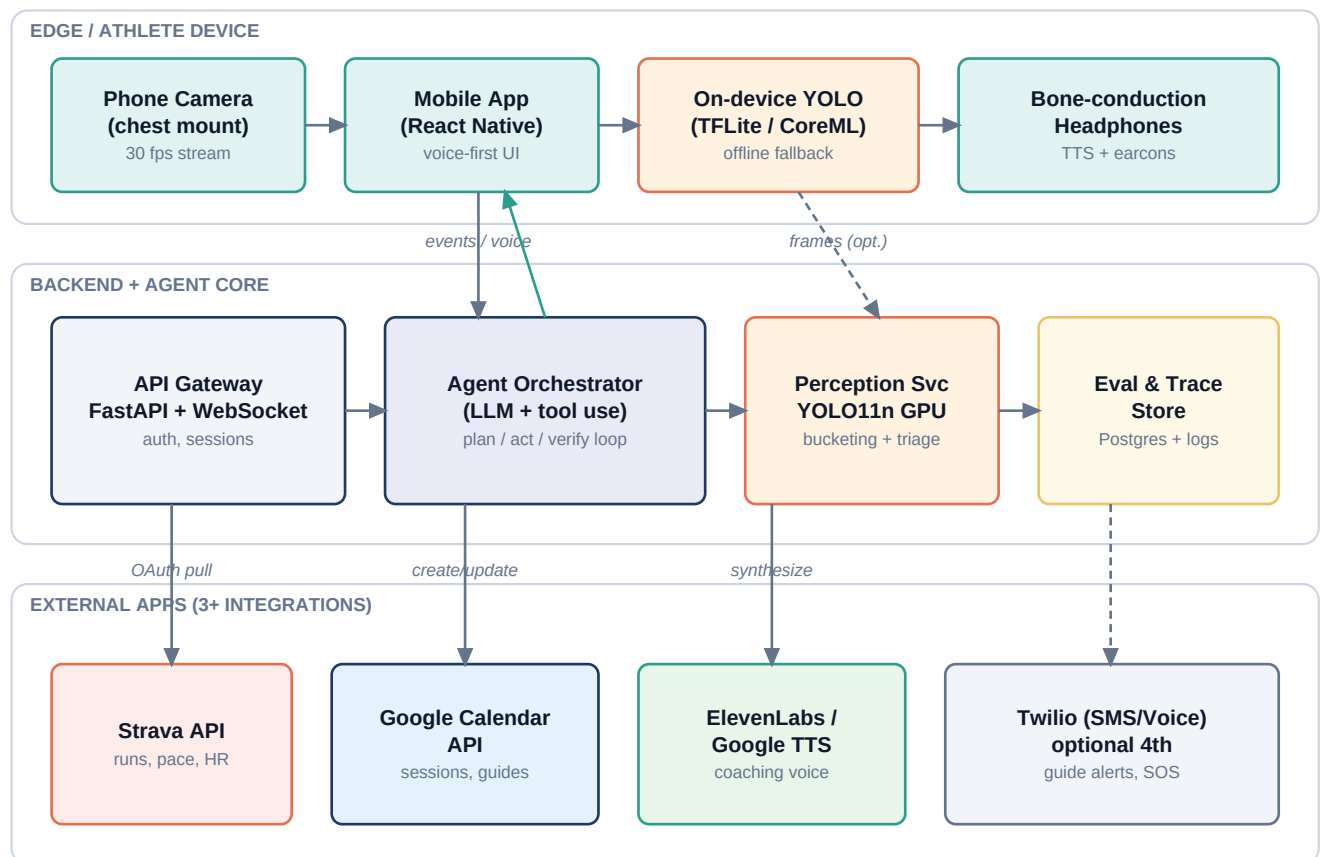


Figure 1 - Three-tier architecture: athlete device, agent core, external apps. Solid arrows are the demo path; dashed are optional.

Design principles

- **Perception at the edge, reasoning in the cloud.** Obstacle alerts must never depend on network latency, so YOLO runs on the phone (TFLite/CoreML) with a GPU server path only for the laptop demo and for eval replays.
- **The agent talks to apps through typed tools.** Each integration is a small, testable function (*strava.get_activities*, *calendar.create_session*, *tts.say*). The LLM never sees raw HTTP; it sees schemas.
- **Every action is idempotent and traced.** Calendar writes carry a deterministic key so a retried plan cannot double-book; every tool call is stored with input, output, latency and retries.
- **One event bus.** Alerts, coaching cues and plan updates are all events; the headset just plays whatever the priority queue emits.

Technology stack

Layer	Choice	Why
Mobile app	React Native (Expo) + react-native-vision-camera	One codebase for iOS/Android; frame processors give near-native camera access; Expo speeds a hackathon build.
On-device model	YOLO11n exported to TFLite (Android) / CoreML (iOS), int8	COCO classes already cover person, bicycle, car, dog, bench, pole-like objects. ~10 ms on a modern phone GPU.
Server perception	Python, Ultralytics YOLO11n/s, ByteTrack, OpenCV, CUDA	Same model family server-side for eval replays and the laptop demo; ByteTrack gives IDs and velocity.
Agent core	Python, Claude API with tool use (or any tool-calling LLM), Pydantic tool schemas	Native multi-step tool calling, structured outputs, easy tracing.
Backend	FastAPI + WebSocket, Postgres, Redis (queue)	Async, typed, quick to write; Postgres for traces/eval; Redis for the alert priority queue.
Audio	ElevenLabs (streaming) or Google Cloud TTS; on-device earcons via expo-av	Streaming TTS keeps coaching latency low; earcons (short panned tones) are faster than words for DANGER.
Integrations	Strava API (OAuth2), Google Calendar API (OAuth2), TTS API, optional Twilio	Three real, public, documented APIs the judges can verify.
Observability	OpenTelemetry traces -> Postgres; simple Streamlit eval dashboard	Cheap to build; makes reliability visible.

6. Perception module (computer vision)

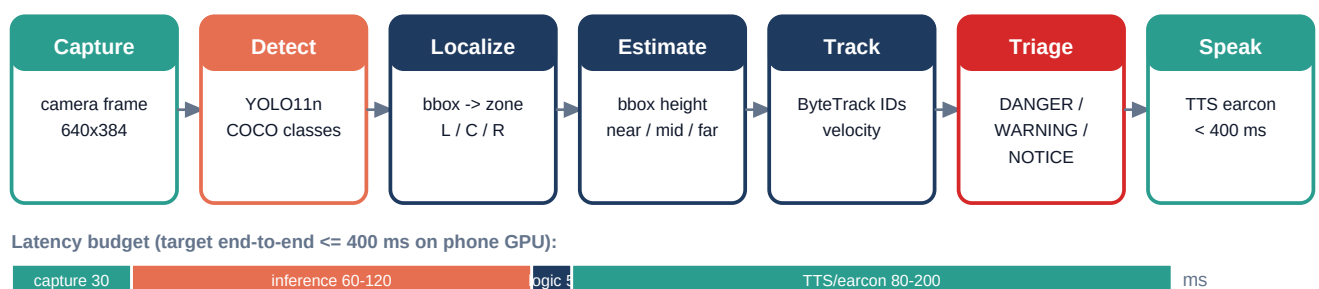


Figure 2 - Seven-stage perception pipeline with latency budget.

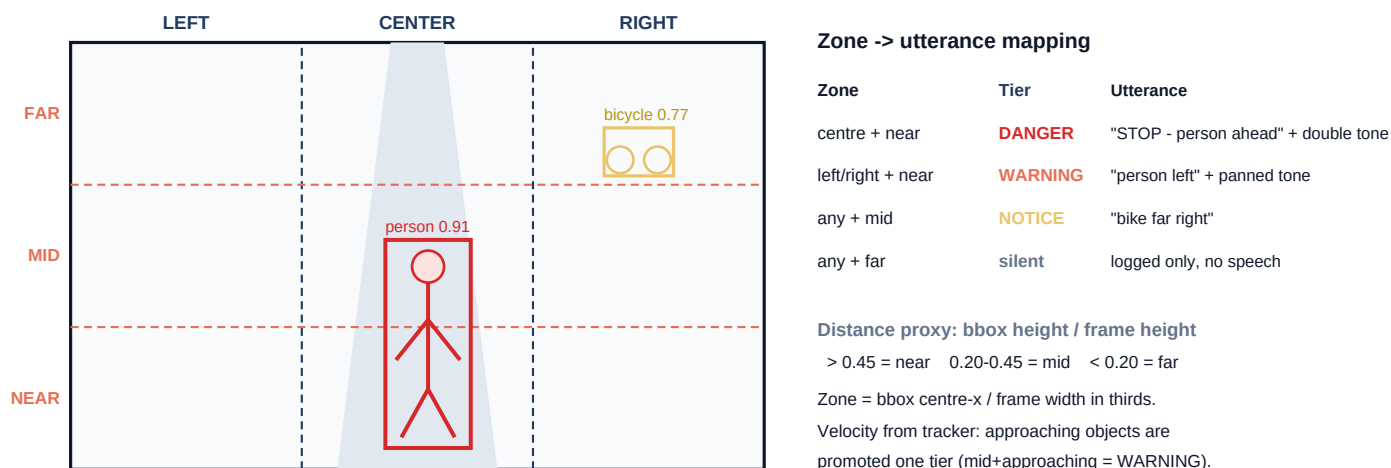


Figure 3 - Spatial bucketing: the frame is split into a 3x3 grid; horizontal thirds give LEFT/CENTER/RIGHT, bounding-box height gives NEAR/MID/FAR.

Model choice and why it is reliable enough

- **YOLO11n (Ultralytics), pretrained on COCO.** No training required. Relevant classes: person, bicycle, car, motorcycle, bus, truck, dog, bench, fire hydrant, stop sign, traffic light, potted plant. Runs locally, no API key, no network - the demo cannot be killed by Wi-Fi.
- **Fallbacks:** Roboflow hosted inference or Google Cloud Vision if the laptop has no GPU (adds ~150-300 ms).
- **Tracking:** ByteTrack (built into Ultralytics via *model.track*) gives stable IDs, so the same person is announced once, not 30 times a second, and approaching objects (bbox height growing frame over frame) are promoted a tier.
- **Depth proxy is a heuristic and we say so.** bbox height / frame height thresholds (0.45 / 0.20) were the fastest honest option; MiDaS monocular depth is a stretch goal if time allows.
- **Debounce and hysteresis:** an object must persist 3 frames before it is announced and must leave for 15 frames before it can be re-announced; DANGER always pre-empts the queue and interrupts current speech.

Triage tiers

Tier	Trigger	Output	Max latency
DANGER	CENTER + NEAR, or any zone NEAR and approaching fast	Double tone + "Stop - person ahead"; interrupts everything	400 ms
WARNING	LEFT/RIGHT + NEAR, or CENTER + MID approaching	Panned single tone (left ear / right ear) + "person left"	600 ms
NOTICE	MID, not approaching	Soft cue, spoken only if queue is idle: "bike far right"	1.5 s
silent	FAR	Logged for eval, never spoken	-

7. Agent core (multi-step orchestration)

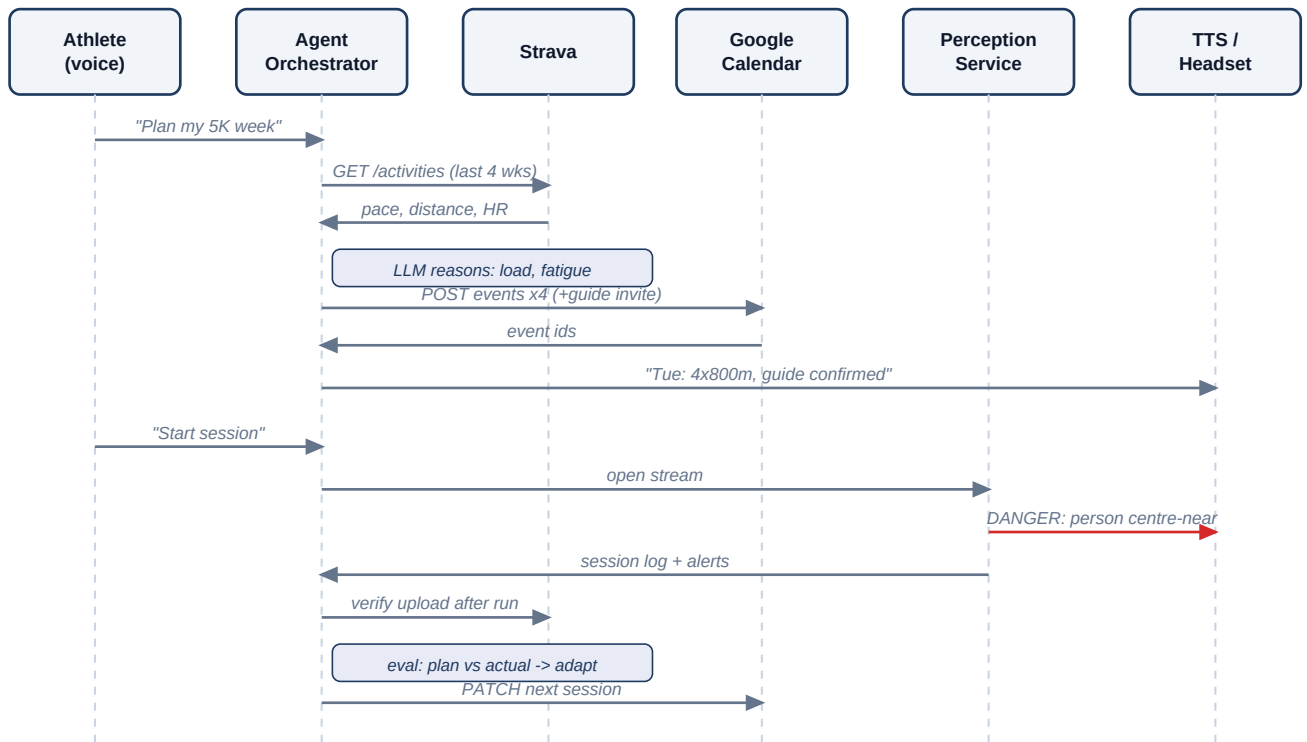


Figure 4 - One full loop: planning a week, running a session with live alerts, verifying and adapting.

Tool inventory exposed to the LLM

Tool	Inputs	Returns	Side effect
strava.get_activities	athlete_id, weeks	list of runs (distance, pace, HR, elevation, date)	none
strava.verify_upload	expected_date, min_km	matched activity or null	none
plan.generate	history, goal, constraints (guide days, venues)	week plan JSON + rationale	none (pure LLM)
plan.validate	week plan	list of rule violations	none
calendar.create_session	date, type, duration, guide_email?, idem_key	event id, invite status	writes Calendar
calendar.check_guide	event id	accepted / declined / pending	none
tts.say	text, priority	audio stream id	plays on headset
session.start / session.end	plan item id	session id / summary (alerts, splits)	starts perception
sms.notify (optional)	guide phone, text	delivery status	Twilio SMS

Orchestration loop

The orchestrator is a plain loop: send state + tools to the LLM, execute the returned tool call, append the result, repeat until the LLM emits a final message. Three guard rails make it reliable: (1) *plan.validate* is mandatory before any calendar write; (2) writes are idempotent by key so retries are safe; (3) a step budget (max 12 calls) and a per-call timeout prevent runaway loops. The system prompt encodes the accessibility constraints - the model cannot schedule a hard outdoor session without a confirmed guide or a safe venue, and every user-facing string must be speakable in one breath.

Planning rules encoded as tests (also used by the eval harness)

- Weekly volume increase capped at 10% versus the trailing 3-week average.
- No two hard sessions on consecutive days; at least one full rest day.
- Hard outdoor session requires `guide.accepted == true OR venue in {track, treadmill}`.

- Every session has a spoken summary under 25 words and a fallback if the guide cancels.

8. External integrations (the 3+ apps)

App	Auth	Endpoints used	Hackathon gotchas
Strava	OAuth2 (scope: activity:read_all)	GET /athlete/activities; GET /activities/{id}/streams (HR, pace)	Rate limit 100 req / 15 min. Create a test athlete and seed it with a few runs the week before. Webhook for upload verification is optional; polling is fine.
Google Calendar	OAuth2 (scope: calendar.events)	events.insert (with attendees + reminders), events.get, events.patch	Use a dedicated Google account; invite the guide's address; poll <code>attendees[].responseStatus</code> for accepted/declined.
ElevenLabs / Google Cloud TTS	API key	text-to-speech streaming	Pre-cache the 12 fixed alert phrases as WAV on the phone; stream only long coaching text.
Twilio (optional 4th)	API key	Messages (SMS), optional voice call for SOS	Trial accounts can only text verified numbers - verify the guide's number in advance.

Why these three: Strava is the **sense** (what actually happened), Calendar is the **act** (commitments with other humans), TTS is the **voice** (the only interface a blind runner can use mid-run). Together they let one agent read, decide, commit and speak.

9. Frontend

The mobile app is deliberately minimal and voice-first. Three screens, all fully navigable with TalkBack / VoiceOver, all with a single large action button.

Screen	Purpose	Accessibility details
Home / Today	Speaks today's session on open; one big button: "Start session". Secondary: "Ask coach" (push-to-talk).	Auto-announces on focus; 64 px touch targets; haptic confirmation; no colour-only state.
Live run	Camera runs in background; screen shows large zone indicator + last alert text for the sighted guide / judges.	All feedback is audio + haptics; screen is a debug view, not the interface.
Week plan	Read-only list of sessions with guide status; long-press to ask the agent to move or change a session.	Each row is one sentence; screen-reader order matches spoken order.

For the judging demo we add a **web dashboard** (Streamlit or Next.js) that mirrors the phone: live detections, the priority queue, the tool-call trace and the eval numbers. Judges watch the dashboard; the runner (or a teammate) hears the phone.

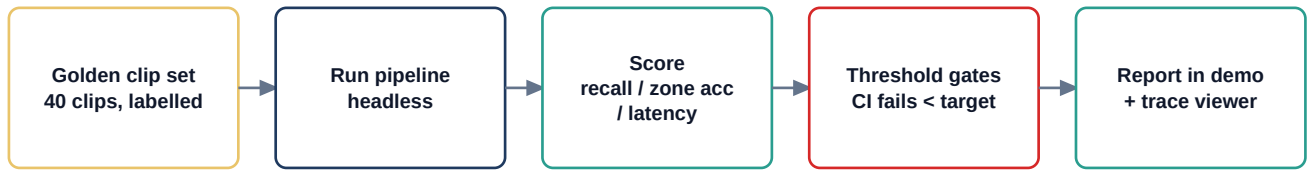
10. Backend and data model

Table	Key fields	Purpose
athlete	id, name, strava_token, calendar_token, guide_contacts[], preferences (venues, cue style)	One row per runner
activity	id, athlete_id, strava_id, date, km, avg_pace, avg_hr, elevation	Cached Strava history
plan_week	id, athlete_id, week_start, plan_json, rationale, version	Versioned so adaptations are visible
session	id, plan_item_id, calendar_event_id, guide_status, started_at, ended_at, summary_json	Links plan to Calendar to what happened
alert	id, session_id, ts, class, zone, distance_bucket, tier, latency_ms, spoken	Every perception event, for eval
tool_trace	id, run_id, step, tool, input_json, output_json, latency_ms, retries, error	The reliability evidence

API surface (FastAPI)

POST /agent/plan-week - POST /agent/chat (push-to-talk text) - POST /session/start - WS /session/{id}/frames (optional server perception) - WS /session/{id}/events (alerts to phone and dashboard) - POST /session/{id}/end - GET /eval/latest

11. Reliability and evaluation plan



Same harness runs on the agent layer: 10 synthetic athlete profiles -> plan sanity rules (no back-to-back hard days, weekly load +<10%)
Every tool call is traced (input, output, latency, retries) and replayable - this is the 'show you know it works' evidence.

Figure 5 - The evaluation harness runs the same pipeline headless against labelled clips and synthetic athletes.

Perception evaluation

- **Golden clip set:** 40 short clips (5-10 s) recorded from a chest-mounted phone while walking/jogging: park path, track, pavement, indoor corridor; day and dusk. Label each clip's obstacles with true zone and bucket at 3 checkpoints (CVAT or a simple JSON file).
- **Metrics:** person recall at NEAR (target ≥ 0.95), zone accuracy (target ≥ 0.90), false DANGER rate per minute (target ≤ 1), end-to-end latency p50/p95 on device (target 250 / 400 ms), duplicate-announcement rate (target 0).
- **Thresholds are gates:** the eval script exits non-zero if a target is missed; we show the passing run in the demo and one earlier failing run to prove the gate is real.

Agent evaluation

- **10 synthetic athletes** (beginner, returning from injury, marathon build, guide only on weekends, treadmill only, etc.) with generated Strava histories.
- **Rule tests** from section 7 run against every generated plan; plus an LLM-as-judge check for "speakable in one breath" and "gives a reason".
- **Adaptation test:** inject a missed session and a cancelled guide; assert the next plan reduces load and swaps venue, and that the athlete is told why.
- **Integration tests** with mocked Strava/Calendar for idempotency (same plan twice -> same events), retry on 429/5xx, and token refresh.

Live reliability in the demo

- Kill the network mid-run: alerts keep working (on-device model), coaching text degrades to cached phrases, dashboard shows the reconnect.
- Force a Calendar 500 on first attempt (feature flag): the trace shows the retry and a single event created.
- Replay a stored session trace step by step so judges can inspect any decision.

12. Safety, ethics and known limitations

- **RunSense is an assistive layer, not a guide replacement.** The app states this on first launch and the agent will not schedule an untethered outdoor session. This is both the honest position and the one expert judges will respect.
- **Depth is a heuristic.** Monocular bbox size can be fooled by children, seated people, or objects at the edge of the frame. We report the failure modes we saw in the clip set.
- **Model bias:** COCO detectors under-detect some body types and low-light scenes; recall is measured per condition and low-light is flagged as unsupported in v1.
- **Privacy:** frames are processed on device and never stored by default; only detection metadata is logged. Server-side frames are used only for the eval set with the teammates' consent.

- **Data minimisation:** Strava and Calendar tokens are stored encrypted; scopes are read-only except Calendar events.
- **Fail-safe audio:** if the phone overheats or the model stalls for > 2 s, the headset plays a distinct "awareness paused" tone rather than staying silent.

13. 48-hour build plan

When	Track A - Perception (2 people)	Track B - Agent + integrations (2 people)	Milestone
Before the event	Record 40 clips; label 20; export YOLO11n to TFLite; test on phone.	Create Strava + Google test accounts, register OAuth apps, seed 4 weeks of runs, verify TTS keys.	All credentials work; model runs on phone.
Hour 0-6	Camera -> YOLO -> zone/bucket -> console print. Debounce logic.	FastAPI skeleton, tool schemas, Strava + Calendar tools with tests (mocked).	First alert printed; first calendar event created from code.
Hour 6-14	ByteTrack, tiers, earcons, TTS phrases cached; WS events to dashboard.	Orchestrator loop with Claude tool use; plan.generate + validate; 3 synthetic athletes pass.	Phone speaks "person left"; agent plans a week end to end.
Hour 14-24	Eval harness over clips; tune thresholds; latency measurement.	Adaptation flow (missed session, guide cancel); idempotency; retries; traces to Postgres.	Eval report v1 with real numbers.
Hour 24-34	Low-light + edge-case clips; fail-safe tone; on-device profiling.	Mobile screens (Expo), push-to-talk, dashboard (Streamlit).	Full loop on the phone.
Hour 34-42	Freeze model + thresholds; re-run eval; record backup video.	Freeze prompts; write README; polish dashboard.	Feature freeze.
Hour 42-48	Rehearse demo x3 with network kill and API-failure flags.	Prepare slides: problem, gap, architecture, eval, limitations.	Submission.

Scope ladder (cut from the bottom if behind): 1) live obstacle alerts on phone, 2) Strava -> plan -> Calendar with guide invite, 3) spoken coaching + adaptation, 4) eval dashboard, 5) Twilio SOS, 6) MiDaS depth.

14. Demo script (5 minutes)

Time	What happens	What judges see / hear
0:00-0:40	Problem in one persona: "Sara is blind, runs 5Ks with a guide, has no coach." Show landscape slide with the gap.	Importance + originality in 40 seconds.
0:40-1:40	Teammate says: "Plan my week, I want a faster 5K, guide only Tue/Sat." Dashboard shows tool calls: Strava pull, reasoning, validate, 4 Calendar events, guide invite.	Multi-step agent + 2 real integrations. Phone speaks the plan.
1:40-3:10	"Start session." Teammate walks with chest phone; another teammate steps in from the left, then straight ahead. Headset (on speaker) says "person left", then double tone + "Stop - person ahead".	Perception + TTS live. Latency counter on dashboard.
3:10-3:50	Kill Wi-Fi: alerts continue. Re-enable. End session; agent verifies Strava upload (seeded) and rewrites Thursday because Tuesday was cut short.	Reliability + adaptation.
3:50-4:40	Eval slide: recall, zone accuracy, false-DANGER rate, latency p95; one failing earlier run; rule-test results for 10 athletes.	"How we know it works".
4:40-5:00	Limitations and what is next (depth, low light, club pilot).	Honesty; clarity.

Backup: a pre-recorded 90-second video of the live segment, in case the venue lighting or audio fails.

15. Resources and references

Models and CV

- Ultralytics YOLO11 docs (detect, track, export to TFLite/CoreML): <https://docs.ultralytics.com>
- ByteTrack paper (Zhang et al., 2022) and Ultralytics tracking guide: <https://docs.ultralytics.com/modes/track>
- COCO class list (80 classes): <https://cocodataset.org>
- react-native-vision-camera frame processors: <https://react-native-vision-camera.com>
- TensorFlow Lite on Android / Core ML on iOS deployment guides (official docs).
- MiDaS monocular depth (stretch goal): <https://github.com/isl-org/MiDaS>

- CVAT for clip labelling: <https://www.cvat.ai>

Integrations

- Strava API reference and OAuth: <https://developers.strava.com/docs/reference>
- Google Calendar API (events.insert, attendees): <https://developers.google.com/calendar/api>
- ElevenLabs streaming TTS: <https://elevenlabs.io/docs> - Google Cloud Text-to-Speech: <https://cloud.google.com/text-to-speech>
- Twilio Messaging quickstart: <https://www.twilio.com/docs/messaging>
- Claude API tool use: <https://docs.claude.com/en/docs/build-with-claude/tool-use>

Backend and eval

- FastAPI + WebSockets: <https://fastapi.tiangolo.com>
- OpenTelemetry Python: <https://opentelemetry.io/docs/languages/python>
- Streamlit for the eval dashboard: <https://streamlit.io>

Domain and prior art

- Google Project Guideline (blog and research write-ups, 2020-2021).
- Google DeepMind Running Guide agent announcement (May 2026) and SG Enable partnership.
- WHO fact sheet on blindness and vision impairment; IAPB Vision Atlas.
- Achilles International and United in Stride (guide-runner matching programmes) - useful for user interviews and demo feedback.
- Paralympic / World Para Athletics guide-runner rules (tether guidance).

Appendix A - Code skeletons

A.1 Perception loop (server / laptop demo)

```
from ultralytics import YOLO
import cv2, time
from alerts import Triage, Speaker

model = YOLO("yolo11n.pt")
tri, spk = Triage(), Speaker()
cap = cv2.VideoCapture(0)
while True:
    ok, frame = cap.read(); t0 = time.time()
    H, W = frame.shape[:2]
    res = model.track(frame, persist=True, conf=0.45, imgsz=640, verbose=False)[0]
    for b in res.bboxes:
        x1, y1, x2, y2 = b.xyxy[0].tolist(); cls = model.names[int(b.cls)]
        tid = int(b.id) if b.id is not None else -1
        cx = (x1 + x2) / 2 / W; h = (y2 - y1) / H
        zone = "left" if cx < 1/3 else "right" if cx > 2/3 else "center"
        dist = "near" if h > 0.45 else "mid" if h > 0.20 else "far"
        tri.observe(tid, cls, zone, dist, h)
    for ev in tri.emit():
        # debounced, prioritised events
        spk.say(ev)
        # earcon + cached phrase, DANGER interrupts
    tri.log_latency((time.time() - t0) * 1000)
```

A.2 Agent orchestrator (tool-use loop)

```
def run_agent(goal, athlete, tools, max_steps=12):
    msgs = [{"role": "user", "content": goal}]
    for step in range(max_steps):
        resp = llm.messages.create(model=MODEL, system=SYSTEM_PROMPT,
                                   tools=[t.schema for t in tools], messages=msgs)
        if resp.stop_reason != "tool_use":
            return resp.content[0].text
        results = []
        for block in resp.content:
            if block.type == "tool_use":
                out = execute(block.name, block.input, athlete) # traced, retried, idempotent
                results.append({"type": "tool_result", "tool_use_id": block.id, "content": out})
        msgs += [{"role": "assistant", "content": resp.content},
                 {"role": "user", "content": results}]
    raise RuntimeError("step budget exceeded")
```

A.3 Plan validation rules

```
def validate(plan, history, guides):
    v = []
    if plan.week_km > 1.10 * trailing_avg_km(history, weeks=3): v.append("volume>+10%")
    for a, b in zip(plan.days, plan.days[1:]):
        if a.hard and b.hard: v.append(f"back-to-back hard {a.date}/{b.date}")
    if not any(d.rest for d in plan.days): v.append("no rest day")
    for d in plan.days:
        if d.hard and d.outdoor and not (guides.accepted(d.date) or d.venue in {"track", "treadmill"}):
            v.append(f"unsafe hard outdoor session {d.date}")
        if len(d.spoken_summary.split()) > 25: v.append(f"summary too long {d.date}")
    return v
```

Appendix B - Risk register

Risk	Likelihood	Impact	Mitigation
Phone too slow for real-time YOLO	Medium	High	int8 TFLite, imgsz 416, skip frames; laptop GPU path as backup.
OAuth / API outage during judging	Low	High	Seeded cache of Strava data; retries; recorded video backup.
Venue lighting kills detections	Medium	Medium	Test at venue; bring a lamp; clip-set includes dim scenes; fail-safe tone.
LLM produces unsafe plan	Medium	High	plan.validate is a hard gate; rule tests in CI; conservative system prompt.
Audio not audible to judges	Medium	Medium	Route headset audio to a speaker; captions on the dashboard.
Team over-scopes	High	High	Scope ladder; feature freeze at hour 42.